Conjecture 00000002520: Separated bases for finite tensor Fourier transforms

GitHub: gaochengzhecpu

Abstract

For any finite number of axes of arbitrary positive lengths, the multidimensional discrete Fourier transform is the tensor product of the one-dimensional transforms. We prove the full forward and inverse formulas, preservation of separability in both directions, and the product formula for an actual Fourier basis. Lean verifies the result for all finite shapes using Mathlib's discrete Fourier linear equivalence, tensor products, tensor-product bases, and canonical coordinate isomorphisms.

1 Scope and precise statement

We read the source's tensor Fourier assertion in the standard setting of finite dimensional complex tensors and the discrete Fourier transform (DFT). The English statement specifies neither a continuous domain nor a function space. Its Chinese text repeats the terms for tensor, high-dimensional Fourier transform, and separated basis, but does not supply an additional coherent mathematical condition. The precise finite discrete statement below is the interpretation proved here. We make no claim about Fourier integrals on Euclidean spaces, infinite tensor products, or analytic convergence questions.

Let $d \geq 0$ and let $N_1, \ldots, N_d$ be arbitrary positive integers. Put

$$G = \prod_{i=1}^{d} \mathbb{Z}/N_i\mathbb{Z}, \qquad \mathcal{V} = \mathbb{C}^G, \qquad M = \prod_{i=1}^{d} N_i.$$

The empty product G is a singleton, so $d = 0$ gives $\mathcal{V} = \mathbb{C}$ and $M = 1$. For each axis use the standard additive character

$$\chi_i(a) = \exp(2\pi \mathrm{i}\, a/N_i),$$

where any integer representative of $a \in \mathbb{Z}/N_i\mathbb{Z}$ gives the same value. Define

$$(\mathcal{F}f)(k) = \sum_{j \in G} \left(\prod_{i=1}^{d} \chi_i(-j_i k_i) \right) f(j).$$

For $f_i : \mathbb{Z}/N_i\mathbb{Z} \to \mathbb{C}$, a separated array is $f(j) = \prod_i f_i(j_i)$.

Theorem 1. *For every finite shape $(N_1, \ldots, N_d)$:*

1. *$\mathcal{F}$ is a complex linear equivalence. Its inverse is*

$$(\mathcal{F}^{-1}g)(j) = \frac{1}{M} \sum_{k \in G} \left(\prod_{i=1}^{d} \chi_i(j_i k_i) \right) g(k).$$

2. *The transform factors as the tensor product of the one-dimensional DFTs under the canonical identification* $\bigotimes_{i=1}^{d} \mathbb{C}^{\mathbb{Z}/N_i\mathbb{Z}} \simeq \mathcal{V}$. *This identity holds for every tensor, whether separated or not. In particular,*

$$\mathcal{F}\left(\prod_i f_i(j_i)\right)(k) = \prod_i (\mathcal{F}_i f_i)(k_i), \qquad \mathcal{F}^{-1}\left(\prod_i g_i(k_i)\right)(j) = \prod_i (\mathcal{F}_i^{-1} g_i)(j_i).$$

3. *The vectors*

$$b_k(j) = \frac{1}{M} \prod_{i=1}^{d} \chi_i(j_i k_i), \qquad k \in G,$$

form a basis of $\mathcal{V}$. *Each vector is the product of its one-dimensional Fourier basis vectors* $N_i^{-1}\chi_i(j_i k_i)$, *and the coordinates of* f *in this basis are exactly* $(\mathcal{F}f)(k)$.

The normalization is deliberate: the forward transform is unnormalized, and the inverse carries $1/M$. Thus $b_k = \mathcal{F}^{-1}\delta_k$, where δ_k is the standard coordinate vector. Axis length one and the empty shape are included.

2 Proof

Proof. The one-dimensional DFT on $\mathbb{Z}/N\mathbb{Z}$ is a complex linear equivalence with inverse

$$(\mathcal{F}_N^{-1} g)(j) = N^{-1} \sum_{k \in \mathbb{Z}/N\mathbb{Z}} \chi(jk) g(k).$$

Apply this equivalence along each axis. These operations are linear equivalences on the full array space, so their composition is again one. Expanding the successive finite sums and interchanging their order gives exactly the defining formula for $\mathcal{F}$. Reversing the composition and expanding the one-dimensional inverse formulas gives the displayed inverse, with normalization $\prod_i N_i^{-1} = 1/M$.

For a separated input, the summand factors into independent terms:

$$\left(\prod_i \chi_i(-j_i k_i)\right) \prod_i f_i(j_i) = \prod_i \big(\chi_i(-j_i k_i) f_i(j_i)\big).$$

Finite distributivity gives

$$\sum_{j \in G} \prod_i \big(\chi_i(-j_i k_i) f_i(j_i)\big) = \prod_i \sum_{j_i \in \mathbb{Z}/N_i\mathbb{Z}} \chi_i(-j_i k_i) f_i(j_i).$$

This is the forward separability formula. The same calculation with positive characters and the product inverse normalization proves the inverse separability formula.

To identify the operator on actual tensor products, suppose the first axis has length N and the remaining grid is G'. The canonical coordinate equivalence

$$T : \mathbb{C}^{G'} \otimes_{\mathbb{C}} \mathbb{C}^{\mathbb{Z}/N\mathbb{Z}} \longrightarrow \mathbb{C}^{(\mathbb{Z}/N\mathbb{Z}) \times G'}$$

sends $g \otimes f$ to the array $f(j_1)g(j')$. The separability formula shows

$$T \circ (\mathcal{F}_{G'} \otimes \mathcal{F}_N) = \mathcal{F}_{(N,G')} \circ T$$

on pure tensors. Pure tensors span the tensor product and both sides are linear, so the identity holds for every tensor. Induction on the number of axes proves the full factorization. The first-axis versus remaining-axis order above only uses the canonical permutation of tensor coordinates. The inverse identity follows by taking inverses.

Finally, $\mathcal{F}^{-1}$ sends the coordinate basis (δ_k) to a basis (b_k). Substituting δ_k into the inverse formula gives the displayed expression for b_k. Factoring the expression proves its product-basis form. Equivalently, the canonical tensor coordinate equivalence carries the tensor product of the remaining-axis Fourier basis and the first-axis Fourier basis onto the full Fourier basis. Since $\mathcal{F}b_k = \delta_k$, the Fourier values are precisely the basis coordinates. This proves every assertion, including the empty shape where all transforms are the identity. □

3 Formal verification and scope

The Lean model records a shape as a list of natural numbers n_i and uses the positive axis length $N_i = n_i + 1$. Every finite list of positive lengths has this form; this encoding is also proved. Its recursive grid is a singleton for the empty shape and $\mathrm{ZMod}(n+1) \times G'$ when an axis is added. The arrays $G \to \mathbb{C}$ are the usual coordinate spaces of finite tensors.

The transform `dftAll` is built from Mathlib's actual `ZMod.dft` linear equivalence, currying, and pointwise linear equivalences on remaining axes. It is not a symbolic operator postulated to satisfy separation. The proof then derives the complete character sum formulas for arbitrary arrays. Mathlib's `ZMod.stdAddChar` is the standard cyclic-group Fourier character used above.

The formalization contains actual Mathlib tensor products and bases. The canonical coordinate equivalence uses

`TensorProduct.piScalarRight`.

The theorem

`TensorFourier.tensor_factorization`

proves the operator identity for every element of the genuine tensor product, by tensor-product induction. Its inverse is proved separately. The theorem

`TensorFourier.tensor_product_basis`

identifies Mathlib's actual `Basis.tensorProduct` with the full Fourier basis under those coordinates. The dimension is arbitrary in all these statements; no fixed two-dimensional example substitutes for the general claim.

The universal proposition proved by

`TensorFourier.conjecture`

includes the forward and inverse character sums, forward and inverse pure-factor separation, the full product-basis identity, actual tensor-product operator factorization, and tensor-product basis compatibility. All parts of Theorem 1 in the stated finite discrete setting are checked. No continuous Fourier-transform assertion is claimed or left implicit.

The strict check treats warnings as errors. The principal theorem audits contain only `propext`, `Classical.choice`, and `Quot.sound`. There are no unproved placeholders or added axioms. Development and self-review were performed by a delegated AI agent; the coordinating agent reviews the work separately. No external independent review is claimed.

4 Source and reproduction

The frozen bilingual statement is preserved in `SOURCE.md`. The verification folder records its provenance and SHA-256. The source is conjecture 00000002520 in the competition repository. Both languages were inspected, and the scope decision is stated explicitly above.

The project pins Lean 4.19.0 and Mathlib commit

`c44e0c8ee63ca166450922a373c7409c5d26b00b`.

From `lean/`, obtain the pinned dependencies and Mathlib cache, then run `lake build` and

`lake env lean -DwarningAsError=true Main.lean`.

The file ends with explicit axiom audits. Compile this paper with Tectonic or another LaTeX engine supporting the listed packages.